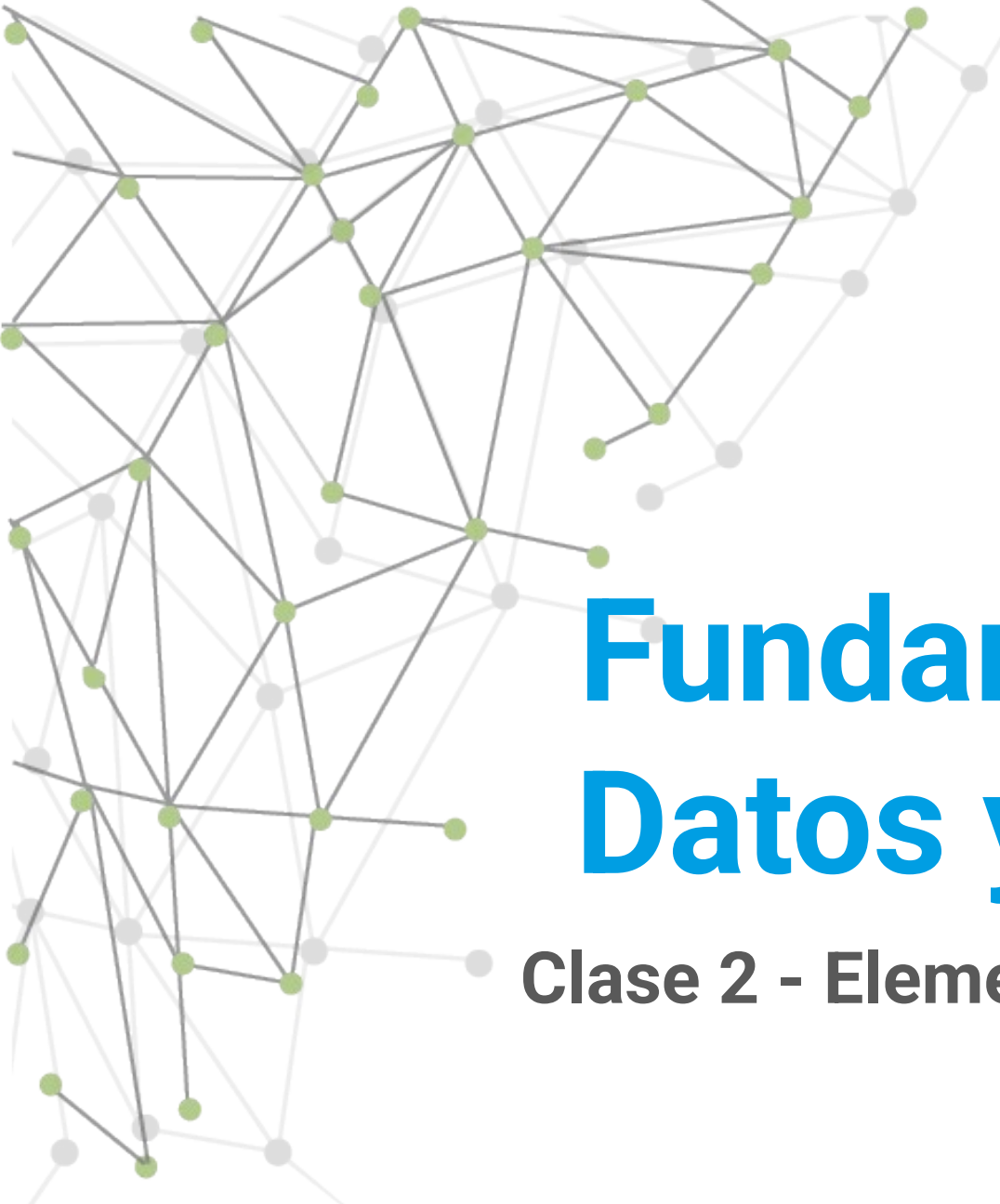




# Fundamentos de Bases de Datos y Consultas Básicas

Clase 2 - Elementos conformantes de una base de datos



# ¿Qué aprenderás en esta sesión?

*Aprenderemos a identificar los elementos fundamentales que componen una base de datos, incluyendo su definición, los distintos tipos de usuarios, permisos, estructuras como tablas y vistas, y funcionalidades avanzadas como procedimientos almacenados y funciones.*

*Además, comprenderemos cómo estos elementos se utilizan en entornos productivos reales como sistemas de gestión empresarial, plataformas de comercio electrónico o entornos logísticos.*



## Desafío inicial

Te integras a un equipo de desarrollo en una empresa de logística. Te entregan acceso a una base de datos pero no puedes modificar información ni comprender del todo qué datos ves o por qué hay código embebido en ciertas consultas. No sabes quién tiene permisos para qué, ni por qué hay estructuras que parecen iguales pero se comportan diferente.

¿Cómo está organizada esta base de datos y qué tipo de elementos la componen?

Para resolverlo deberás: Comprender qué es una base de datos, identificar los distintos elementos que la componen, reconocer cómo se estructuran los permisos y analizar funciones y procedimientos usados para automatizar tareas.

# ¿Qué es una base de datos?

## ¿Por qué necesitamos una base de datos si ya contamos con archivos de Excel?

En el mundo laboral actual, las bases de datos son esenciales para operar con grandes volúmenes de información. Desde un sistema bancario que gestiona millones de transacciones diarias, hasta una empresa de transporte que debe coordinar rutas, vehículos y clientes, todas dependen de sistemas robustos, seguros y organizados. Las bases de datos son el pilar de estos sistemas, permitiendo un acceso rápido, seguro y organizado a los datos.

En entornos industriales, por ejemplo, se utilizan bases de datos para rastrear inventarios, controlar maquinaria o registrar eventos de mantenimiento. En sectores de servicios, una base de datos puede almacenar el historial de atención al cliente, resultados de encuestas, o facturación automatizada. La trazabilidad, eficiencia y control que otorgan las bases de datos es fundamental para la toma de decisiones basada en evidencia.

# Definición y propósito

Una base de datos (BD) es un sistema organizado de almacenamiento de información que permite guardar, consultar, modificar y eliminar datos de forma estructurada, consistente y segura. Los sistemas gestores de bases de datos (DBMS, por sus siglas en inglés) como PostgreSQL, MySQL o SQL Server son las herramientas que permiten esa interacción.

Además de almacenar datos, una base de datos permite definir relaciones entre entidades (tablas), establecer reglas (restricciones) y garantizar la integridad de la información. Es decir, no se trata solo de guardar información, sino de hacerlo bajo ciertas condiciones que mantengan la calidad, disponibilidad y seguridad de los datos en entornos reales, colaborativos y con acceso simultáneo.

# Tipos de bases de datos

| Tipo de BD         | Característica principal                                                     | Casos de uso reales                     |
|--------------------|------------------------------------------------------------------------------|-----------------------------------------|
| Relacional (SQL)   | Datos organizados en tablas con relaciones definidas. Requiere esquema fijo. | CRM, sistemas contables, bancos.        |
| Documental (NoSQL) | Usa documentos tipo JSON. Flexible y escalable.                              | Aplicaciones móviles, catálogos online. |
| Clave-valor        | Registra pares clave/valor. Rápido y simple.                                 | Caches, configuración de sistemas.      |
| Grafos             | Representa nodos y relaciones.                                               | Redes sociales, recomendaciones.        |
| Columnar           | Datos almacenados por columnas. Optimiza consultas analíticas.               | BI, Big Data (ej. Apache Cassandra).    |

Tabla 1. Tipos comunes de bases de datos y su aplicación productiva. Fuente: Desafío Latam

# Buenas prácticas y errores comunes

## Buenas prácticas

- Elegir el tipo de base de datos según el caso de uso, no por tendencia.
- Evaluar la frecuencia de lectura/escritura y la escalabilidad necesaria.
- Priorizar sistemas robustos y bien documentados para ambientes de producción.

## Errores comunes

- Implementar un modelo relacional en sistemas NoSQL, lo cual genera ineficiencia.
- Usar bases relacionales sin normalizar.
- Elegir tecnologías sin considerar la experiencia del equipo o comunidad de soporte.

# Diferencias con otras estructuras de datos

A diferencia de un archivo Excel, una base de datos permite la separación lógica entre la estructura (modelo de datos) y el contenido (datos). Incluye mecanismos de control de concurrencia, seguridad, validación y respaldo.

| Criterio                                    | Hoja de cálculo        | Base de datos relacional   |
|---------------------------------------------|------------------------|----------------------------|
| Escalabilidad                               | Limitada               | Alta                       |
| Integridad referencial                      | No                     | Sí                         |
| Control de accesos                          | Limitado o inexistente | Granular por usuario y rol |
| Transacciones seguras                       | No                     | Sí (ACID)                  |
| Soporte para múltiples usuarios simultáneos | No                     | Sí                         |

Tabla 2. Comparación entre hoja de cálculo y base de datos relacional.

Fuente: Desafío Latam

# Buenas prácticas y errores comunes

## Buenas prácticas

- Usar hojas de cálculo solo para prototipos o visualización rápida.
- Adoptar BD cuando los datos necesitan control de acceso, validación, persistencia o integridad.

## Errores comunes

- Usar Excel para compartir datos entre áreas, provocando descontrol de versiones.
- Subestimar los riesgos de pérdida o corrupción en archivos no gestionados.

# Ejemplo técnico (PostgreSQL)

```
-- Crear una base de datos básica  
CREATE DATABASE gestion_clientes;
```

Este comando genera una nueva base en blanco sobre la cual se podrán crear tablas, definir usuarios y ejecutar consultas. Es el primer paso antes de construir cualquier sistema basado en datos.

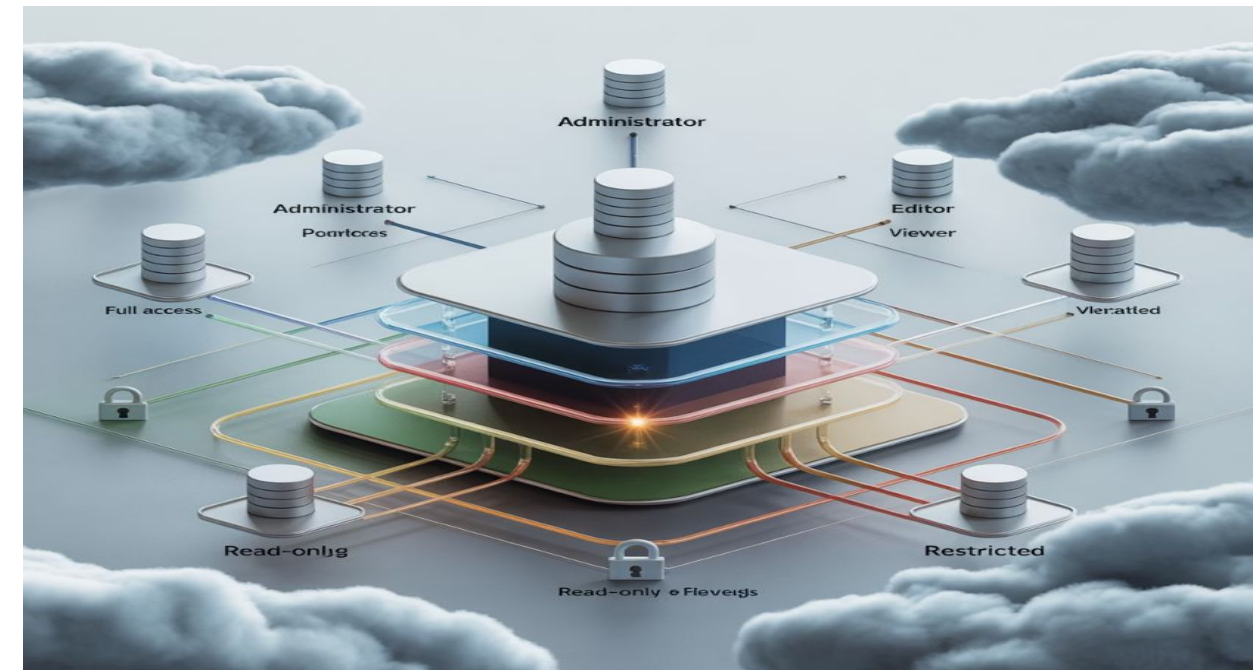
Ahora que comprendemos qué es una base de datos, sus tipos y diferencias clave respecto de otras herramientas, estamos en condiciones de analizar cómo se gestiona el acceso a estos datos y quién puede hacer qué dentro del sistema. Esto lo exploramos a continuación mediante el análisis de usuarios y permisos.

# Usuarios y permisos

¿Qué consecuencias tendría que una persona sin experiencia pueda eliminar datos históricos de clientes en una base de datos productiva?

## Rol de los usuarios en el entorno técnico-profesional

Las bases de datos deben ser administradas cuidadosamente para que distintos perfiles puedan cumplir sus funciones sin poner en riesgo la información. En un sistema de gestión hospitalaria, por ejemplo, el personal médico podría consultar historiales clínicos, pero no debería tener permiso para modificar estructuras de la base. En cambio, un administrador del sistema sí puede modificar estas estructuras, pero no necesariamente ver datos sensibles.



En el marco de la gestión de datos profesional, se definen roles con privilegios específicos que permiten segmentar el acceso a las funciones de consulta, inserción, modificación o eliminación de información. Estos roles permiten aplicar principios de seguridad informática como el principio de "mínimo privilegio necesario".

# Tipos de roles y permisos

El uso de roles permite agrupar permisos y asignarlos a múltiples usuarios de forma sencilla y controlada. Los sistemas DBMS permiten asignar y revocar permisos en tiempo real, manteniendo control sobre la auditoría de accesos.

| Rol           | Permisos asignables                                           | Contexto real de uso           |
|---------------|---------------------------------------------------------------|--------------------------------|
| Administrador | Crear/eliminar objetos, asignar roles, ver datos sensibles    | DBA, coordinador de TI         |
| Desarrollador | Crear tablas, insertar datos de prueba, modificar estructuras | Área técnica y de desarrollo   |
| Analista      | Leer datos, consultar vistas, exportar información            | Área de BI, calidad o finanzas |
| Auditor       | Leer todos los registros sin alterar nada                     | Seguridad o cumplimiento       |

Tabla 3. Roles y permisos habituales en entornos organizacionales.

Fuente: Desafío Latam

# Buenas prácticas y errores comunes

## Buenas prácticas

- Crear roles por función (ej. solo lectura, lectura-escritura, administrador).
- Documentar los cambios de permisos en cada sprint de desarrollo.
- Asignar permisos a roles, no a usuarios directamente.

## Errores comunes

- Uso de la instrucción GRANT ALL sin criterio.
- Uso compartido de cuentas de acceso.
- Falta de trazabilidad en modificaciones de permisos.

# Ejemplo:

Para garantizar que las personas accedan sólo a la información relevante a su función, es necesario usar instrucciones SQL para otorgar y revocar permisos. Supongamos que existe una tabla empleados, y queremos permitir a quien tenga el rol de analista solo consultar los datos:

```
-- Otorgar permiso de Lectura
GRANT SELECT ON empleados TO analista_rrhh;

-- Asegurarse de que no puede modificar datos
REVOKE INSERT, UPDATE, DELETE ON empleados FROM analista_rrhh;
```

Este ejemplo garantiza que el acceso está restringido únicamente a la operación de lectura (SELECT), evitando la posibilidad de que se modifiquen datos sensibles como salarios o datos personales.

La gestión de usuarios y permisos es una práctica esencial para proteger la información en entornos profesionales. Una vez definidos los accesos, es necesario entender cómo se organiza internamente esa información: qué estructuras se usan para almacenarla (tablas) y para mostrarla de forma parcial o filtrada (vistas). Este será el enfoque del próximo apartado.

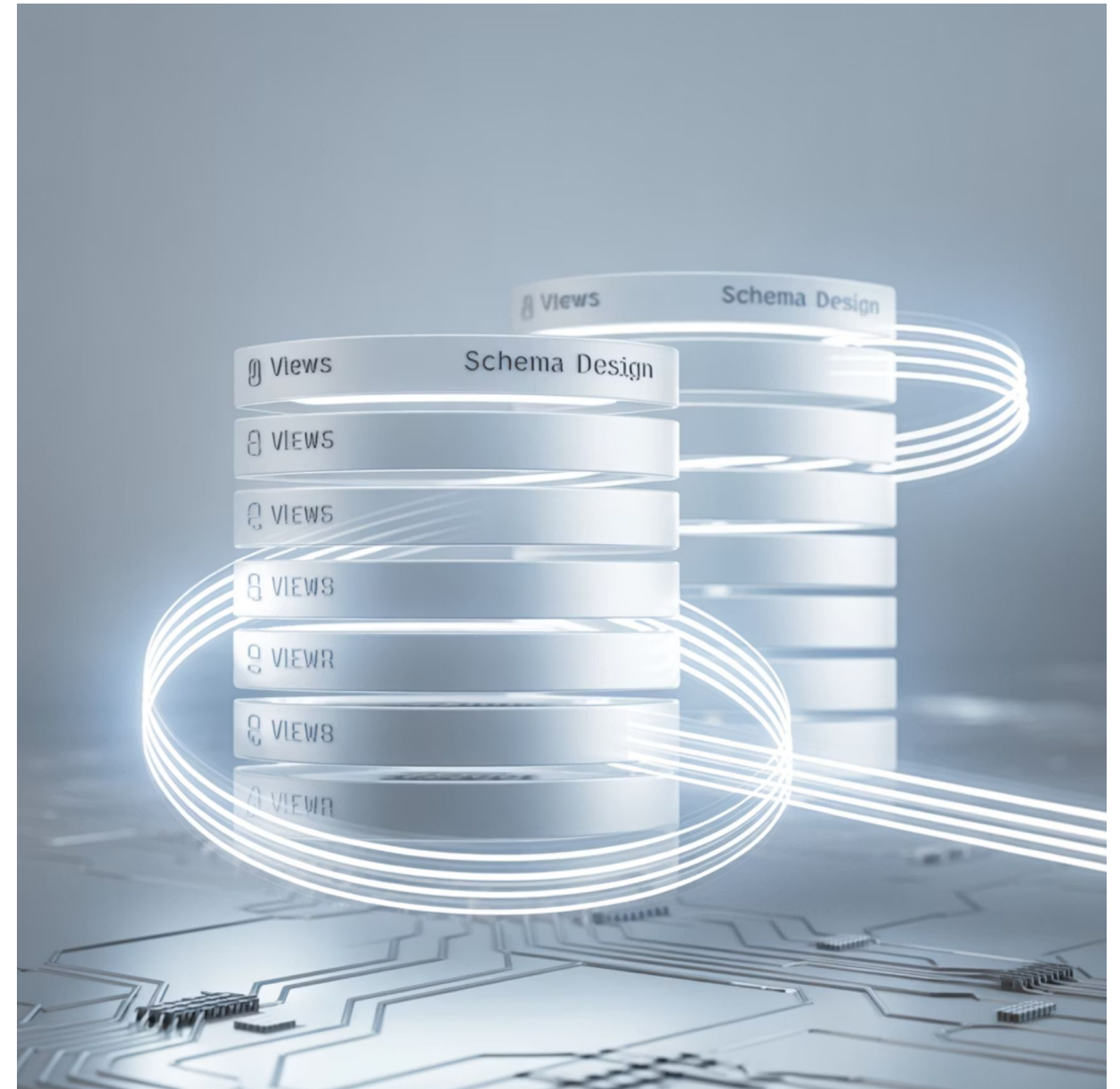
# Tablas y vistas

¿Por qué sería riesgoso entregar acceso directo a todas las tablas de una base de datos a cualquier usuario, sin filtrar la información que puede ver?

## Contexto productivo: uso de tablas y vistas en entornos reales

En sistemas empresariales, las tablas son la estructura fundamental donde se almacenan datos operacionales, como clientes, productos o transacciones. Las vistas, por otro lado, se utilizan para restringir o simplificar el acceso a la información. Por ejemplo, una tabla puede contener todos los datos de clientes, pero una vista mostrará solo los clientes activos y omitirá datos sensibles como el RUT o número de tarjeta de crédito.

Tanto las tablas como las vistas permiten estructurar la información de manera lógica y segura. Comprender sus diferencias y usos es clave para construir sistemas eficientes, escalables y seguros.



# Tablas: estructura física del almacenamiento

Una tabla es una estructura que organiza datos en columnas (campos) y filas (registros). Cada tabla debe tener una clave primaria (primary key) que identifique de forma única a cada fila. Las columnas tienen tipos de datos definidos, como enteros, texto o fechas.

## Ejemplo aplicado:

```
CREATE TABLE empleados (  
    id SERIAL PRIMARY KEY,  
    nombre TEXT NOT NULL,  
    departamento TEXT,  
    salario NUMERIC CHECK (salario > 0)  
);
```

Esta tabla almacena los registros de personas empleadas. Define un identificador único (id), restringe nombre como obligatorio (NOT NULL) y valida que el salario sea mayor a cero.

# Buenas prácticas y errores comunes

## Buenas prácticas

- Utilizar nombres claros y en minúsculas para tablas y columnas.
- Establecer claves primarias y restricciones desde el diseño inicial.
- Aplicar normalización para evitar duplicación innecesaria de datos.

## Errores comunes

- No definir una clave primaria.
- Permitir nulos en columnas que deberían ser obligatorias.
- Usar nombres ambiguos o inconsistentes.

# Vistas: estructura lógica de consulta

Las vistas son consultas guardadas que permiten mostrar información específica de una o más tablas. No almacenan datos, pero actúan como filtros dinámicos. Sirven para limitar el acceso, simplificar consultas complejas o facilitar reportes.

## Ejemplo:

```
CREATE VIEW empleados_activos AS
SELECT
    nombre,
    departamento
FROM empleados
WHERE salario > 0;
```

Esta vista muestra solo nombre y departamento de personas con salario positivo, ocultando otras columnas.

| Criterio       | Tabla                 | Vista                           |
|----------------|-----------------------|---------------------------------|
| Almacena datos | Sí                    | No                              |
| Editable       | Sí                    | Parcialmente (según definición) |
| Uso principal  | Almacenamiento físico | Consultas, seguridad, reportes  |

Tabla 4. Comparación entre tabla y vista.

# Buenas prácticas y errores comunes

## Buenas prácticas

- Crear vistas para exponer solo la información necesaria.
- Documentar cada vista con su propósito y estructura.
- Actualizar vistas si cambian las tablas base.

## Errores comunes

- Usar vistas en exceso sin revisar el rendimiento.
- Pensar que las vistas siempre mejoran la eficiencia.
- Intentar modificar datos desde vistas complejas no actualizables.

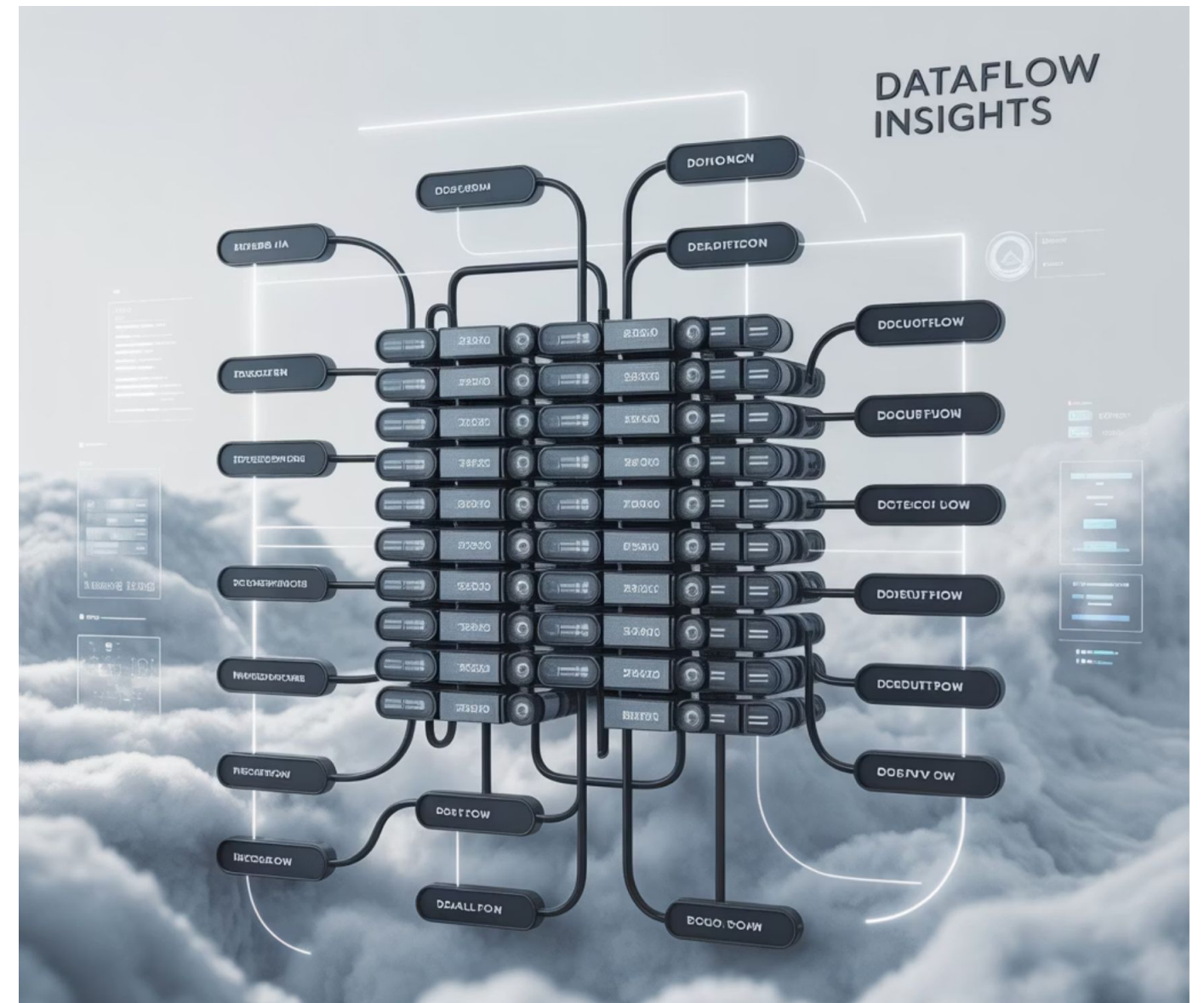
Ya comprendemos cómo se estructuran y presentan los datos usando tablas y vistas. Ahora veremos cómo automatizar tareas recurrentes o complejas dentro de la base de datos mediante procedimientos almacenados y funciones, elementos claves en entornos que exigen eficiencia y control de procesos.

# Procedimientos almacenados y funciones

¿Existe alguna forma de automatizar tareas dentro de una base de datos, como calcular descuentos, generar alertas o registrar auditorías, sin depender de código externo?

Los procedimientos almacenados (stored procedures) y las funciones permiten encapsular lógica de negocio directamente dentro del sistema gestor. Esto es especialmente útil en organizaciones que requieren eficiencia y control, como instituciones financieras, clínicas o plataformas con múltiples transacciones por segundo.

Por ejemplo, en un sistema de gestión académica, un procedimiento puede inscribir automáticamente a una persona en varios cursos; una función puede calcular su promedio final.



# Procedimientos almacenados

Un procedimiento almacenado es un conjunto de instrucciones SQL que se ejecutan como una sola unidad. Puede aceptar parámetros, modificar datos y no requiere retornar un valor.

## Ejemplo:

```
CREATE PROCEDURE registrar_empleado(  
    n TEXT,  
    d TEXT,  
    s NUMERIC  
)  
LANGUAGE plpgsql AS $$  
BEGIN  
    INSERT INTO empleados (nombre, departamento, salario)  
    VALUES (n, d, s);  
END;  
$$;  
  
-- Llamado del procedimiento  
CALL registrar_empleado('Luis', 'Ventas', 850000);
```

# Funciones

A diferencia de los procedimientos, las funciones retornan un valor y pueden usarse dentro de consultas SQL. Son útiles para cálculos, validaciones o transformaciones.

## Ejemplo:

```
CREATE FUNCTION calcular_bono(s NUMERIC)
RETURNS NUMERIC AS $$
BEGIN
    RETURN s * 0.10;
END;
$$ LANGUAGE plpgsql;

-- Ejemplo de uso
SELECT calcular_bono(1000000); -- Retorna 100000
```

# Buenas prácticas y errores comunes

## Buenas prácticas

- Dividir responsabilidades: funciones para cálculo, procedimientos para acciones.
- Usar nombres descriptivos y coherentes.
- Documentar los parámetros y retorno esperado.

## Errores comunes

- Omitir manejo de errores.
- No declarar tipos de retorno.
- Repetir lógica en múltiples partes del sistema.

Los procedimientos almacenados y funciones permiten automatizar reglas de negocio y tareas repetitivas dentro del propio sistema de base de datos, mejorando el rendimiento, reduciendo errores humanos y centralizando la lógica crítica.

# Actividad guiada: Explorando la estructura de una base de datos organizacional



# Objetivo de la actividad

Aplicar conocimientos adquiridos para identificar, analizar y manipular los elementos conformantes de una base de datos, incluyendo tablas, vistas, usuarios, permisos, procedimientos almacenados y funciones, en un entorno controlado que simula una base de datos real de una empresa.



# Situación problema

Formas parte del equipo de analistas de una empresa de servicios que está auditando su sistema de base de datos.

Te han solicitado revisar la estructura actual, verificar qué usuarios tienen acceso a qué datos, identificar qué vistas están disponibles para el área de análisis, y evaluar la automatización actual implementada mediante funciones y procedimientos.

Se te entregará un script con la estructura básica de la base de datos y deberás:

- Identificar todas las tablas, sus campos y claves primarias.
- Explorar las vistas disponibles y deducir su propósito.
- Revisar los permisos asignados a distintos roles.
- Ejecutar y analizar el comportamiento de un procedimiento almacenado y una función.



# Instrucciones

Ejecuta el siguiente script base para crear la estructura de trabajo:

```
-- Crear tabla principal
CREATE TABLE empleados (
  id SERIAL PRIMARY KEY,
  nombre TEXT NOT NULL,
  area TEXT,
  salario NUMERIC CHECK (salario > 0)
);

-- Insertar algunos datos
INSERT INTO empleados (nombre, area, salario) VALUES
  ('Ana', 'TI', 950000),
  ('Luis', 'Ventas', 800000),
  ('Marta', 'Marketing', 920000);

-- Crear vista para analistas
CREATE VIEW vista_empleados_publicos AS
SELECT
  nombre,
  area
FROM empleados;
```

```
-- Crear función para calcular bono
CREATE FUNCTION calcular_bono(s NUMERIC)
RETURNS NUMERIC AS $$
BEGIN
  RETURN s * 0.05;
END;
$$ LANGUAGE plpgsql;

-- Crear procedimiento para registrar empleados
CREATE PROCEDURE registrar_empleado(
  n TEXT,
  a TEXT,
  s NUMERIC
)
LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO empleados (nombre, area, salario)
  VALUES (n, a, s);
END;
$$;

-- Crear roles (simulado)
-- En motores como PostgreSQL usar CREATE ROLE, aquí
se simula asignación de permisos
```

# Instrucciones (continuación)

Verifica la estructura: ¿qué campos tiene la tabla?, ¿qué información muestra la vista?, ¿qué hace la función?, ¿qué parámetros usa el procedimiento?

Ejecuta un ejemplo con la función y el procedimiento:

```
SELECT
    s.nombre_sucursal,
    p.nombre_producto,
    SUM(v.cantidad) AS unidades_vendidas
FROM ventas v
JOIN productos p ON v.id_producto = p.id_producto
JOIN sucursales s ON v.id_sucursal = s.id_sucursal
GROUP BY s.nombre_sucursal, p.nombre_producto;
```

**Documenta:**

¿Qué datos están disponibles para las personas con acceso a la vista? ¿Qué impacto tendría permitir acceso total a la tabla en vez de usar vistas? ¿Qué ventaja representa encapsular la lógica en funciones o procedimientos?

**{desafío}**  
**latam\_**

## Materiales necesarios

- Entorno de ejecución SQL local o en línea (PostgreSQL, SQLite, DBeaver, pgAdmin, etc.).
- Script base proporcionado por el relator.
- Editor de texto o cuaderno para reflexiones.

# Resumen



Comprendimos qué es una base de datos, sus tipos y principales diferencias con otras estructuras de datos.



Exploramos la estructura y función de tablas y vistas.



Identificamos los distintos tipos de usuarios y cómo se asignan permisos mediante roles.



Aprendimos cómo los procedimientos almacenados y funciones permiten automatizar tareas y centralizar la lógica de negocio.



## Próxima sesión...

*En la siguiente sesión abordaremos el modelo de datos y su estructura.*

*Aprenderemos a identificar los elementos fundamentales de un modelo de datos (entidades, atributos e identificadores) y a representar relaciones entre tablas mediante diagramas entidad-relación, herramienta clave para diseñar bases de datos relacionales robustas y comprensibles desde el inicio de un proyecto.*

# Preguntas de cierre

¿Qué diferencia práctica existe entre una tabla y una vista, y cuándo se recomienda usar cada una?

¿Qué rol cumple una función en la automatización de cálculos dentro de una base de datos?

¿Qué beneficios aporta definir restricciones (como CHECK o NOT NULL) en la creación de una tabla?

¿Por qué es importante definir roles y no asignar permisos directamente a usuarios individuales?

¿Qué impacto podría tener modificar o eliminar una función usada en múltiples procesos del sistema?

# Referencias bibliográficas

- Coronel, C., & Morris, S. (2019). Database Systems: Design, Implementation, & Management (13th ed.). Cengage Learning.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/>
- Date, C. J. (2019). An Introduction to Database Systems (8th ed.). Pearson Education.